

AI-Powered Deep Packet Inspection & Network Intrusion Detection System

Complete Interview Preparation Notes

PART 1 — PROJECT SUMMARY

1.1 What is this project? (The 30-Second Pitch)

"I built an end-to-end Deep Packet Inspection (DPI) engine in Python that parses raw network traffic (PCAP files) at the byte level, tracks connections, identifies applications (even inside encrypted HTTPS using TLS SNI), and blocks traffic based on rules. I then extended it with a Machine Learning layer: I engineered 86 statistical flow features, trained Random Forest and XGBoost models on the industry-standard CIC-IDS2017 intrusion detection dataset (~2.8 million flows, 8 attack classes), achieving 99.7% accuracy and 0.9999 ROC-AUC. Finally, I deployed the model as a model-agnostic Flask REST API containerized with Docker."

1.2 Project Timeline / Two Phases

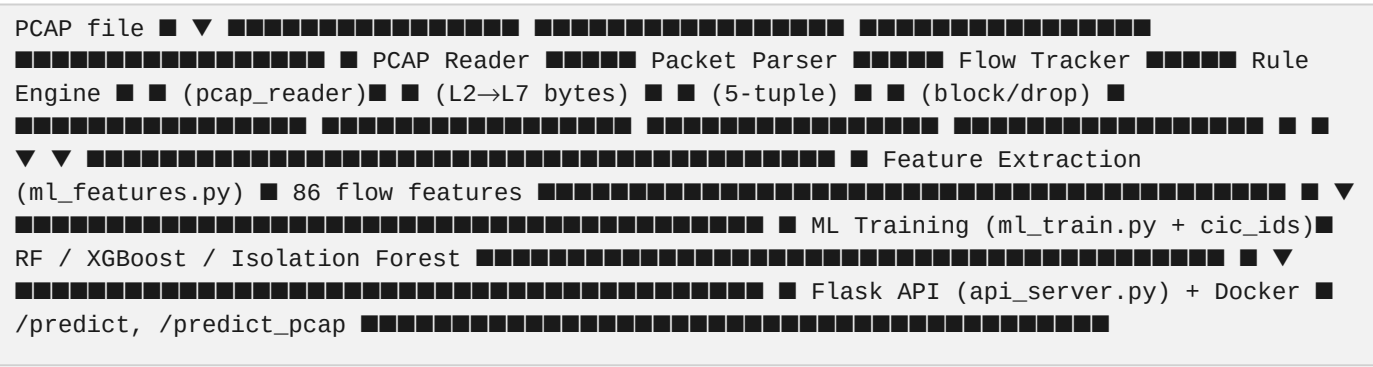
Phase	What was built	Output
Phase 1: Rule-based DPI Engine	PCAP reader, packet parser (Ethernet/IPv4/TCP/UDP), flow tracker, SNI/HTTP/DNS extractors, rule engine, CLI with blocking options	<code>main.py</code> — analyze PCAP, classify apps, block by IP/port/domain/app
Phase 2: AI/ML Redesign	Flow feature extraction (86 features), ML training pipeline, CIC-IDS2017 integration, evaluation framework, Flask API, Docker	<code>ml_pipeline.py</code> , <code>api_server.py</code> — train, evaluate, deploy models

Why the ML phase? Rule-based DPI relies on string matching (SNI/domains). It fails when traffic is encrypted without SNI (VPNs, DoH, ESNI), when new apps appear, or for zero-day attacks. ML learns patterns from flow statistics (packet sizes, timing, directions) and detects anomalies — no signature needed.

1.3 Tech Stack

- **Language:** Python 3.11
- **Libraries:** `struct` (binary parsing), NumPy, pandas, scikit-learn (Random Forest, Isolation Forest, metrics), XGBoost, Flask, pickle (model serialization), SHAP (interpretability)
- **Dataset:** CIC-IDS2017 (Canadian Institute for Cybersecurity, ~2.8M flows, 8 normalized classes, 79 CICFlowMeter features)
- **Deployment:** Docker (python:3.11-slim), Flask REST API
- **Formats:** PCAP (classic), TLS/HTTP/DNS parsing at L2–L7

1.4 Architecture Overview (High-Level)



PART 2 — KEY COMPONENTS (Explain Like You Built It)

2.1 PCAP Reader (`pcap_reader.py`)

- Reads the **PCAP file format**: 24-byte global header (magic `0xa1b2c3d4`, version, snaplen, network/link type) followed by per-packet 16-byte headers + raw bytes.
- Handles **byte order** (endianness): magic `0xd4c3b2a1` means the file is byte-swapped; all headers use network byte order (big-endian) via Python `struct` with `!` prefix.
- Yields `(packet_header, raw_data)` tuples, parsed one at a time — **streaming**, so even large captures don't blow memory.

2.2 Packet Parser (`packet_parser.py`)

Parses by **peeling encapsulation layers from the outside in** using `struct.unpack`:

Layer	Format	Key fields
Ethernet (14 B)	<code>!6s6sH</code>	dst MAC, src MAC, EtherType (0x0800 = IPv4)
IPv4 (20–60 B)	<code>!BBHHHBBH4s4s</code>	IHL (lower nibble × 4 = header len), protocol (6=TCP, 17=UDP), TTL, src/dst IP
TCP (20–60 B)	<code>!HHIIBBHHH</code>	ports, seq/ack, data offset (upper 4 bits × 4), flags (SYN=0x02, ACK=0x10, FIN=0x01, RST=0x04), window
UDP (8 B)	<code>!HHHH</code>	ports, length, checksum

Key detail to mention: **IHL = (version_ihl & 0x0F) * 4** and **TCP header length = (data_offset >> 4) * 4** — these let us find exactly where the application payload begins.

2.3 Flow Tracker (`flow_tracker.py`)

- A **network flow** = one **bidirectional conversation**, identified by the **5-tuple**: `(src_ip, dst_ip, src_port, dst_port, protocol)`.

- Because server→client packets have swapped src/dst, I keep a **reverse index**: when a packet's tuple isn't found, look up its reversed tuple → both directions merge into ONE flow object.
- **State machine**: NEW → ESTABLISHED → CLASSIFIED (or BLOCKED) → CLOSED
- SYN→ESTABLISHED, SYN-ACK→ESTABLISHED, FIN or RST→CLOSED.
- **Why flows matter**: classification (SNI extraction) happens ONCE per flow, then the result is cached for all subsequent packets — huge performance win vs inspecting every packet.
- **Resource management**: idle flows (no packets for 300 s) are evicted; flow table capped at 100,000 flows.
- Per-flow stats collected: packet/byte counts per direction, packet lengths, inter-arrival times, TCP flag counts, initial window size — **these become the ML features**.

2.4 Application Identification (DPI techniques in `sni_extractor.py`, `flow_tracker.py`)

Classification order (only first packet in flow): 1. **TLS SNI (port 443)**: parse the TLS Client Hello — content type 0x16 (handshake), handshake type 0x01; walk the extensions (TLV) to find SNI extension (type 0x0000) → plaintext hostname. This is how we identify HTTPS destinations even though the payload is encrypted. 2. **HTTP Host header (port 80)**: look for `Host:` line in plaintext payload. 3. **DNS queries (port 53)**: parse length-prefixed label names (e.g., `3www6google3com0`). 4. **Port-based fallback**: map well-known port → app type.

Domain→app classification uses **case-insensitive substring matching** (e.g., `youtube/ytimg` → YOUTUBE, `facebook/fbcdn` → FACEBOOK).

2.5 Rule Engine (`rules.py`)

Four rule types: **block by IP, port, domain, app category**. Rules can be passed via CLI (`--block-ip 1.2.3.4 --block-domain youtube.com --block-app SOCIAL_MEDIA`) or loaded from a rules file. Blocked flows are marked BLOCKED and their packets are **dropped** from the output PCAP (packets of allowed flows are written to a filtered output file). Stats track forwarded vs dropped.

2.6 CLI (`main.py`)

```
python main.py input.pcap --block-domain youtube.com --block-app SOCIAL_MEDIA --output filtered.pcap
```

Produces a report: total packets, TCP/UDP split, forwarded/dropped, flows, SNI found, app classification histogram, top domains.

2.7 Feature Extraction (`ml_features.py`) — THE ML CORE

`extract_flow_features()` converts a FlowState (sequence of packets) into a **fixed-length vector of 86 numerical features**, grouped as:

1. **Basic flow info** — duration, destination port, protocol
2. **Counts** — fwd/bwd packets & bytes, totals
3. **Packet length stats** — min/max/mean/std/median/percentiles (10/25/75/90) for fwd & bwd
4. **Inter-arrival time stats** — same statistics on gaps between packets (per direction + combined)
5. **Ratios** — packet ratio, byte ratio, down/up ratio

6. **TCP flags** — SYN/FIN/RST/PSH/ACK/URG/CWE/ECE counts + ratios
7. **Payload entropy** — Shannon entropy of payload bytes: encrypted/compressed data $\approx$ 7.5–8.0, HTTP text $\approx$ 4.0–5.5, DNS $\approx$ 3.0–4.5 → distinguishes encrypted vs plaintext without decryption
8. **TLS version** (encoded 0–3), **rates** (bytes/sec, packets/sec), **active/idle times** (gaps > 1.0 s = idle)

Why 86 features and not just 10? Attack traffic differs from benign in *statistical* patterns (e.g., DoS has many small packets with tiny IATs, portscan has high SYN ratio, botnets have regular periodic IATs). Distributions matter more than single numbers — hence mean AND std AND percentiles.

2.8 ML Training (`ml_train.py`, `ml_pipeline.py`, `cic_ids.py`)

Data pipeline: labeled PCAPs → DPI engine processes packets → flows built → features extracted → pandas DataFrame → train/test split (80/20, stratified when possible) → StandardScaler → train.

Models implemented: | Model | Why chosen | Key params | |---|---|---| | Random Forest | Handles mixed/continuous features, robust, interpretable, parallelizes | 200 trees, max_depth 20, class_weight='balanced' | | XGBoost | State-of-the-art for tabular data, handles missing values | 200 rounds, lr 0.1, subsample 0.8 | | Logistic Regression | Fast baseline, interpretable | multinomial, balanced weights | | Gradient Boosting | Non-linear patterns, accuracy priority | 200 estimators, depth 6 | | Isolation Forest (anomaly) | Detects zero-day/unknown traffic — anomalies are easier to isolate (shorter tree paths) | contamination 0.1 |

Class imbalance handling (crucial): BENIGN is ~80% of CIC-IDS2017; INFILTRATION has only 36 samples. Used `class_weight='balanced'` (scikit-learn auto-weights classes inversely to frequency) + CLI options for **undersampling**, **SMOTE**, and **SMOTE-ENN** oversampling.

Results (Random Forest on CIC-IDS2017, held-out test — real numbers from `evaluation/cic_rf_model_eval.json`): - Accuracy: **99.74%** - ROC-AUC (macro): **0.99996** - F1 (weighted): **0.9979**, F1 (macro): 0.907 - Test set: 565,576 flows - Note: macro metrics are lower because rare classes (INFILTRATION: 7 test samples) hurt macro averaging — a good honest point to make in interviews.

2.9 Model Artifacts (`models/`)

Each saved model (pickle) bundles: `model`, `scaler`, `metadata` (model type, feature count, label names, training/test counts, accuracy), `feature_names/feature_columns`, and `label_encoder` (for CIC models) — this is what makes the **API model-agnostic**.

2.10 Flask API (`api_server.py`) + Docker

Endpoints: | Endpoint | Purpose | |---|---| | GET /health | Health check, model info, uptime | | GET /features | List features the loaded model expects | | POST /predict | Predict label from JSON flow features | | POST /predict_pcap | Upload a PCAP → engine extracts flows → predicts per flow, returns table with confidence | | GET /stats | API usage statistics | | GET /dashboard | HTML dashboard |

Model-agnostic design: the server reads `feature_names`, `label_encoder`, and `label_names` from the model file itself, so ANY trained model (custom DPI or CIC-IDS2017) works without code changes. Features are auto-scaled with the saved scaler; predictions include confidence = max probability.

Docker: `python:3.11-slim` base, installs `requirements.txt`, exposes port 5000, runs `api_server.py`.

2.11 Evaluation Assets (`evaluation/`)

Confusion matrix, ROC curves, feature importance, correlation heatmap, SHAP summary plot, class distribution plots — I generated these to *understand* the model, not just report accuracy.

PART 3 — INTERVIEW Q&A

Section A: Project Introduction (You Will Be Asked This First)

Q1. Tell me about your project. Give the 30-second pitch from section 1.1, then say: "It has two parts — a rule-based DPI engine and an ML-based classifier deployed as an API. Would you like me to go deeper into either?"

Q2. Why did you build this? What problem does it solve? Organizations need to know what applications run on their network and block unwanted traffic — for bandwidth control, policy enforcement, and security. Traditional port-based identification fails because apps hide behind port 80/443. DPI inspects payloads (SNI) to see the real app. Rule-based approaches can't detect unknown/zero-day attacks, so I added ML that learns from statistical flow patterns.

Q3. What was your role? I designed and implemented the full pipeline end-to-end: packet parsing, flow tracking, feature engineering, model training and evaluation, and API deployment. (If mentor-guided: "I built it under a mentor's guidance — I wrote all the code, they reviewed and guided the design.")

Q4. What was the biggest challenge? Honest answer: **class imbalance on CIC-IDS2017** — 2.3M benign flows vs 36 infiltration samples. Overall accuracy was misleading (a model predicting "benign" always would hit 80%). I handled it with balanced class weights, stratified splits, macro-averaged metrics, and evaluated per-class confusion matrices instead of trusting accuracy alone.

Section B: Networking Concepts

Q5. What is the OSI model and where does your project sit? 7 layers: Physical, Data Link, Network, Transport, Session, Presentation, Application. My parser works from Layer 2 (Ethernet) through Layer 7 (HTTP/TLS/DNS). DPI itself is a Layer 7 technique — inspecting payload content beyond port numbers.

Q6. What is a network packet? What's in an Ethernet/IP/TCP header? Packet = header(s) + payload, like a letter: envelope = L2/L3 headers, content = application data. Ethernet: dst/src MAC + EtherType. IPv4: version+IHL, total length, protocol, TTL, src/dst IP. TCP: ports, seq/ack, data offset, flags, window. (See table in section 2.2 — recite it confidently.)

Q7. What is the TCP three-way handshake? Why SYN/ACK/FIN/RST? Client→SYN→Server, Server→SYN+ACK→Client, Client→ACK→Server. SYN opens, ACK acknowledges, FIN closes gracefully, RST aborts abruptly. My flow tracker uses these flags to transition flow states (SYN→ESTABLISHED, FIN/RST→CLOSED).

Q8. What is a 5-tuple? Why is it important? (src IP, dst IP, src port, dst port, protocol) — uniquely identifies a connection. It's the dictionary key for my flow table and the basis for consistent hashing in distributed DPI (all packets of a flow must go to the same worker).

Q9. What is endianness? Why does it matter for packet parsing? Network byte order is big-endian; x86 is little-endian. Protocol headers are defined in network byte order, so I use `struct.unpack('!...')` — the `!` tells Python to convert. PCAP files may also store their magic in swapped order (`0xd4c3b2a1`), which signals the file itself is byte-swapped.

Q10. What is SNI? How does DPI see it even though HTTPS is encrypted? Server Name Indication is an extension in the TLS Client Hello that carries the target hostname in **plaintext** — it's sent before encryption

starts so the server can pick the right certificate. DPI parses the Client Hello's extension list (TLV format) and reads the hostname without decrypting anything.

Q11. How would you identify applications if SNI were encrypted (ESNI/DoH/VPN)? ML flow statistics: packet size distributions, inter-arrival times, byte ratios, payload entropy — which is exactly why I built the ML layer. Encrypted traffic has high entropy and characteristic timing/size fingerprints.

Q12. What is entropy and why did you use it as a feature? Shannon entropy measures unpredictability of bytes. Encrypted/compressed data $\approx 7.5\text{--}8.0$ bits/byte; plaintext HTTP $\approx 4.0\text{--}5.5$; DNS $\approx 3.0\text{--}4.5$. It distinguishes encrypted from plaintext without decryption.

Section C: DPI Implementation Details

Q13. Why track flows instead of inspecting each packet? Three reasons: (1) classification (SNI extraction) is expensive — do it once, cache per flow; (2) blocking decisions are consistent — don't forward 3 packets then block the 4th; (3) you get aggregate statistics (duration, sizes, counts) that per-packet inspection can't give and that ML needs.

Q14. How do you handle bidirectional flows? Reverse-index lookup: `five_tuple.reversed()` — if packet's tuple isn't in the flow table, check the reversed tuple. This merges client→server and server→client into one flow object with forward/reverse counters.

Q15. How do you handle many flows / memory? Idle timeout (300 s) evicts stale flows on periodic cleanup; `MAX_FLOWS = 100,000` with oldest-eviction when exceeded. This mirrors production firewalls (e.g., conntrack).

Q16. What happens to packets that match a block rule? They're dropped — not written to the output PCAP; the flow is marked BLOCKED and all subsequent packets of that flow are dropped by cached decision. Statistics track dropped vs forwarded.

Q17. Why did you parse with struct instead of Scapy? Two reasons: (1) learning — byte-level parsing teaches you the protocol formats, header offsets, and endianness; (2) performance and control — pure Python struct parsing is lightweight, dependency-free, and I control every field. (Honest note: Scapy is great for prototyping.)

Section D: ML Questions (Expect Deep Diving Here)

Q18. Walk me through your ML pipeline end to end. Labeled PCAPs → DPI engine builds flows → extract 86 features per flow → pandas DataFrame, clean Inf/NaN → stratified 80/20 split → StandardScaler → train RF/XGBoost (class_weight balanced) → evaluate (accuracy, precision/recall/F1 macro+weighted, confusion matrix, ROC-AUC) → save model+scaler+metadata as pickle → load in Flask API → predict with confidence.

Q19. Why Random Forest? Why XGBoost? RF: ensembles of decision trees → low variance, handles mixed continuous features, no scaling needed, parallel, interpretable via feature importance. XGBoost: gradient boosting with regularization → state-of-the-art on tabular data, handles missing values, often better accuracy. I compared both and RF delivered 99.7% accuracy on CIC-IDS2017.

Q20. What is the difference between classification and anomaly detection in your project? Classification (RF/XGB) answers "which attack type?" — needs labeled data. Anomaly detection (Isolation Forest, One-Class SVM) answers "is this different from normal?" — trained on benign only, detects zero-day attacks. Isolation Forest works by randomly isolating points: anomalies have short isolation paths because they're few and different.

Q21. Why did you use StandardScaler? Tree models don't need it, but I used it for model consistency across algorithms and because feature scales differ wildly (duration in seconds vs byte counts in millions). The

scaler is fitted on training data only (avoiding data leakage) and saved with the model so inference applies the same transform.

Q22. What is class imbalance and how did you handle it? CIC-IDS2017 is dominated by BENIGN (~80%); INFILTRATION has 36 flows. Without treatment, models become biased to the majority class. Handling: `class_weight='balanced'` (weights $\propto 1/\text{frequency}$), stratified train/test splits, optional SMOTE/undersampling, and — most important — evaluating with macro-F1 and per-class confusion matrices rather than accuracy.

Q23. Which features matter most? How do you know? From `feature_importance` plots (RF feature importance, SHAP summary): flow duration, packet length means/std, IAT statistics, SYN count, and payload entropy ranked highest. E.g., portscan flows have high SYN counts and short durations; DoS flows have high packet rates and small, uniform packet sizes. SHAP also shows *direction* of influence (e.g., high duration → benign, high SYN ratio → attack).

Q24. What metrics did you use and why? Accuracy (overall), precision/recall/F1 (macro AND weighted — macro is honest about rare classes), confusion matrix (see which classes get confused), ROC-AUC (threshold-independent ranking quality). For anomaly detection: precision/recall/AUC too.

Q25. How did you avoid data leakage? Scaler fitted only on training data; stratified split before scaling; features extracted per-flow from timestamps only (no future information); train/test split with fixed `random_state=42`; held-out test evaluation; labels assigned from file-level ground truth, not from the model's own DPI classification during CIC training.

Q26. Why pickle for model storage? When would you not use it? Simple, zero-dependency, stores arbitrary Python objects (model + scaler + metadata in one file). Not safe for untrusted sources (arbitrary code execution on `pickle.load`), not portable across Python versions, no schema. Production alternatives: ONNX, MLflow, or joblib at minimum. (Saying this shows maturity.)

Q27. What would you do differently in production? (1) Version data & models (MLflow/DVC); (2) ONNX/Triton serving instead of pickles; (3) monitoring + drift detection (PSI/KL divergence) on live predictions; (4) feature store instead of recomputing per request; (5) retraining pipeline on new labeled captures; (6) vectorized/batch processing or C-accelerated parsing (DPDK, nDPI, Rust) for high throughput; (7) security: schema validation, auth on API.

Q28. What is the difference between your 86 features and CICFlowMeter's 79? They overlap ~50% conceptually (duration, packet length stats, flag counts, IAT stats) but differ in exact fields — e.g., I added payload entropy and TLS version; CICFlowMeter has subflow stats and bulk-transfer rates. I built a feature-mapping layer (`CIC_TO_OUR_FEATURES`) that bridges the two formats, which is why the API can serve both model types.

Q29. Why did you pick CIC-IDS2017? Industry benchmark, free, includes realistic multi-day capture with 14 attack types spanning brute force, DoS, DDoS, web attacks, infiltration, botnet, portscan; and its CSV form (flow features + labels) is directly usable for ML.

Section E: Deployment & Engineering

Q30. Why Flask over FastAPI? Flask is simpler, has a huge ecosystem, and was sufficient for two prediction endpoints. (If asked — I know FastAPI gives automatic OpenAPI docs, async, and pydantic validation; I'd choose it for production.)

Q31. How does `/predict_pcap` work? Uploaded PCAP → saved to temp → `PcapReader` → `PacketParser` → `DPIEngine` builds flows → per-flow feature extraction → scaled → `model.predict_proba` → returns per-flow table: src/dst IP, ports, duration, packet count, SNI, ground truth (if known), prediction, confidence.

Q32. What does "model-agnostic" mean in your API design? The API doesn't hardcode feature names or labels. It reads them from the saved model file (`feature_names`, `label_encoder`, `label_names`) at load time. Swap the model file → the API adapts automatically. This was a design requirement because I serve both custom-DPI and CIC-IDS2017 models.

Q33. How did you test your work? Generated synthetic test PCAPs (`generate_test_pcap.py`) with realistic flows; generated per-attack-type PCAPs (DoS, DDoS, portscan, brute-force, bot, webattack, infiltration) to validate both DPI classification and ML prediction; CLI reports for end-to-end verification; evaluation JSON + plots for ML quality.

Q34. Why Docker? Reproducibility — identical environment (Python 3.11-slim, pinned requirements) on any machine; ease of deployment; isolation. `docker build -t dpi . && docker run -p 5000:5000 dpi`.

Section F: Behavioral / Confidence Builders

Q35. What would you improve given more time? Real-time PCAP capture (not just offline files), multi-threaded pipeline with consistent hashing (design is documented), deep learning models (1D-CNN on raw bytes, autoencoders), model versioning, and a dashboard for live monitoring.

Q36. What did you learn from this project? Concrete networking fundamentals (headers, encapsulation, TCP state machines, TLS handshake), binary parsing, feature engineering from time-series data, honest ML evaluation under class imbalance, and full-stack ML engineering (training → serving).

Q37. Why did you choose Python for this? Fast prototyping, rich ML ecosystem (scikit-learn, XGBoost, pandas), readable code, and adequate performance for offline analysis; the design (flow caching, single classification per flow) keeps it fast.

Q38. What does "DPI" stand for and how is it different from a firewall? Deep Packet Inspection — inspecting payload content (Layer 7) beyond the header fields a traditional firewall looks at. Firewalls filter by IP/port/protocol; DPI identifies applications and content even in encrypted traffic via SNI and statistical analysis.

Q39. Can you explain SHAP and feature importance? Feature importance (RF) = average impurity decrease from splitting on a feature → which features the model relies on. SHAP = game-theoretic attribution of each feature's contribution to each prediction, consistent across models, with direction (positive/negative effect). Summary plot = each point is a sample, colored by feature value.

Q40. What is ROC-AUC? ROC curve plots TPR vs FPR across thresholds; AUC = probability a random positive ranks above a random negative. 0.99996 means near-perfect separation of classes regardless of threshold.

PART 4 — RESUME BULLETS (ML Role)

1. **Engineered 86 statistical flow features** from raw network traffic (packet length distributions, inter-arrival times, TCP flag counts, byte ratios, payload entropy) by building a custom PCAP-to-feature pipeline, and mapped them to CICFlowMeter's 79 standard features for benchmark compatibility.
2. **Trained and optimized Random Forest and XGBoost classifiers on the CIC-IDS2017 dataset** (~2.8M flows, 8 classes), handling severe class imbalance using class weighting/undersampling/SMOTE, achieving **99.7% accuracy, 0.9999 ROC-AUC, 0.998 weighted F1**.
3. **Implemented a complete evaluation framework** producing confusion matrices, per-class precision/recall/F1, ROC curves, feature importance, and SHAP summary plots to identify model strengths and failure modes on rare attack classes.

4. **Deployed the model as a model-agnostic Flask REST API** — auto-loads feature names and label encoders from saved model files, exposes `/predict` (single flow) and `/predict_pcap` (batch analysis) endpoints, containerized with Docker.

PART 5 — KEY NUMBERS TO MEMORIZE

Item	Value
ML features extracted per flow	86
CICFlowMeter features in dataset	79
Dataset size	~2.8M flows, 8 normalized classes, 8 CSV files (~960 MB)
Accuracy (Random Forest, held-out test)	99.74%
ROC-AUC (macro)	0.99996
F1 weighted / F1 macro	0.9979 / 0.907
Test flows in evaluation	565,576
Flow idle timeout	300 s
Max flows	100,000
Random Forest config	200 trees, max_depth 20, class_weight balanced
Ethernet / IPv4-min / TCP-min / UDP header sizes	14 / 20 / 20 / 8 bytes
TCP flags	SYN 0x02, ACK 0x10, FIN 0x01, RST 0x04, PSH 0x08
TLS Client Hello content type	0x16 (handshake), 0x01 = Client Hello, SNI ext type 0x0000
API endpoints	/health, /features, /predict, /predict_pcap, /stats, /dashboard
8 CIC attack classes	BENIGN, DoS, DDoS, PORT_SCAN, BRUTE_FORCE, WEB_ATTACK, BOT, INFILTRATION

PART 6 — QUICK REVISION CHEAT SHEET (Last-Minute)

- **DPI** = inspect payload (L7), not just headers → find real apps behind port 80/443.
- **SNI** = plaintext hostname in TLS Client Hello → DPI sees HTTPS destinations.
- **5-tuple** = identity of a connection → key of flow table, hashed for load balancing.
- **Flow** = bidirectional conversation → state machine, cached classification, ML features.
- **IHL** = (byte & 0x0F) * 4; **TCP data offset** = (byte >> 4) * 4 → where payload begins.

- **Endianness** = network is big-endian → `struct` format !.
- **Entropy** ≈ 8.0 = encrypted; $\approx 4-5$ = plaintext text.
- **Imbalance** → balanced weights, stratified split, macro metrics, per-class confusion matrix.
- **Isolation Forest** = anomalies isolated by short paths.
- **Model-agnostic API** = read features/labels from the pickle, not from code.
- **99.74% accuracy, 0.99996 AUC** on CIC-IDS2017 — always quote these.
- **Improvements:** real-time capture, multithreaded pipeline, ONNX/MLflow, drift monitoring, DL models.

Notes compiled from the actual project codebase (D:\desktop\DPI\Packet_analyzer) — every number above is real and verifiable in `evaluation/cic_rf_model_eval.json` and the source files.